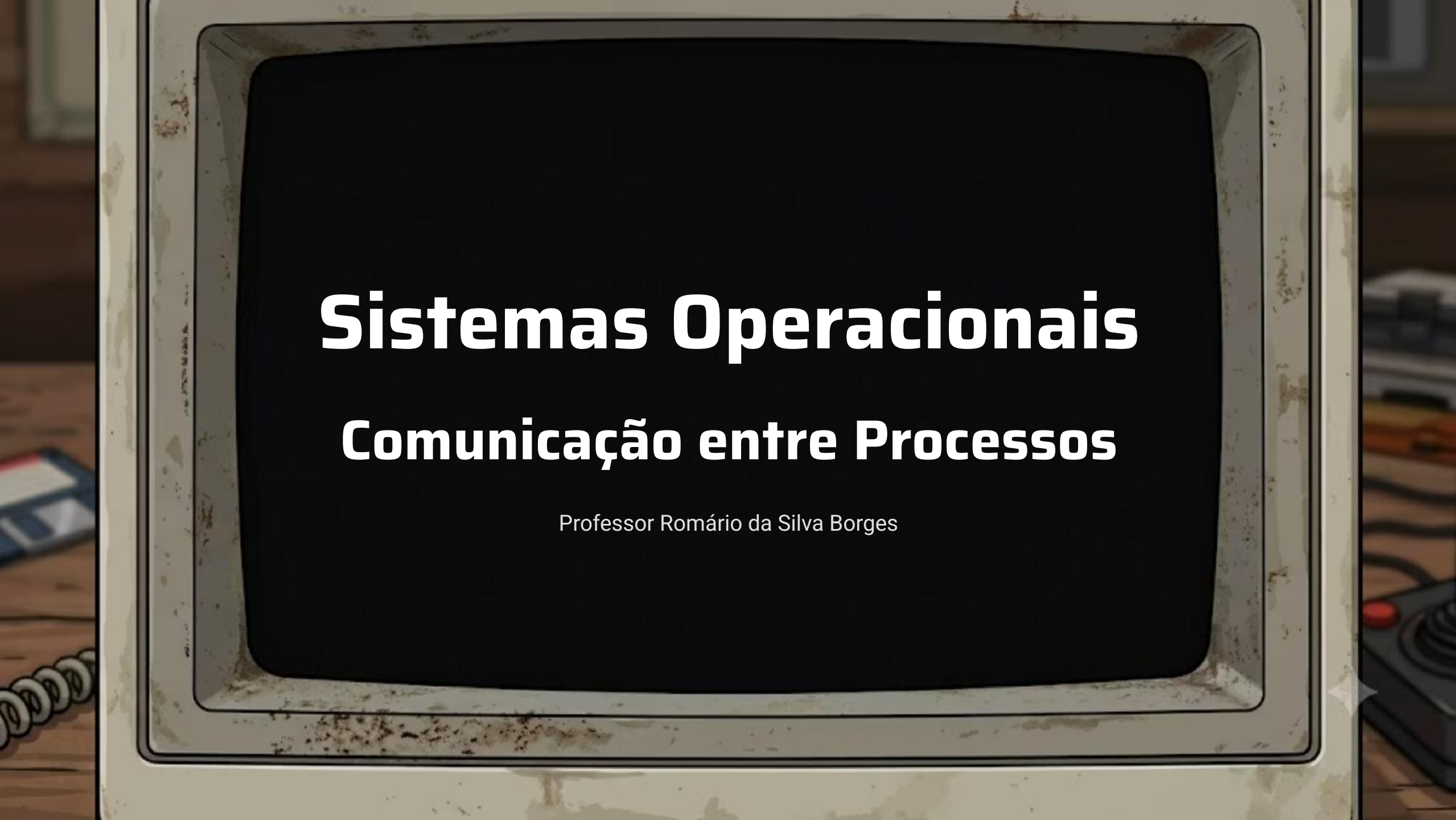


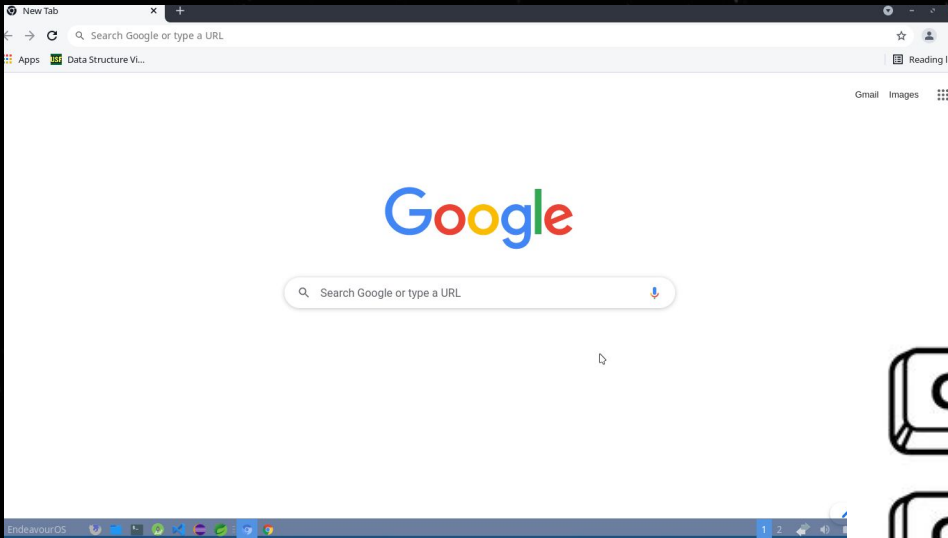


Sistemas Operacionais

Comunicação entre Processos

Professor Romário da Silva Borges

Pense no processo de copiar um texto no navegador e colocar em um navegador... Como acontece essa comunicação?



Ctrl + C

Ctrl + V

shutterstock.com · 1491873533



Por que comunicar processos?



Trocar informações



Compartilhar resultados



Acessar recursos comuns



Cooperar em uma mesma tarefa



Dividir responsabilidades

- Se as **tarefas estão no mesmo processo**, elas **compartilham a mesma área de memória** e a comunicação pode então ser implementada facilmente, usando variáveis globais comuns
- Entretanto, **caso as tarefas pertençam a processos distintos**, não existem variáveis compartilhadas, neste caso, a **comunicação tem de ser feita por intermédio do núcleo do sistema operacional**, usando chamadas de sistema.

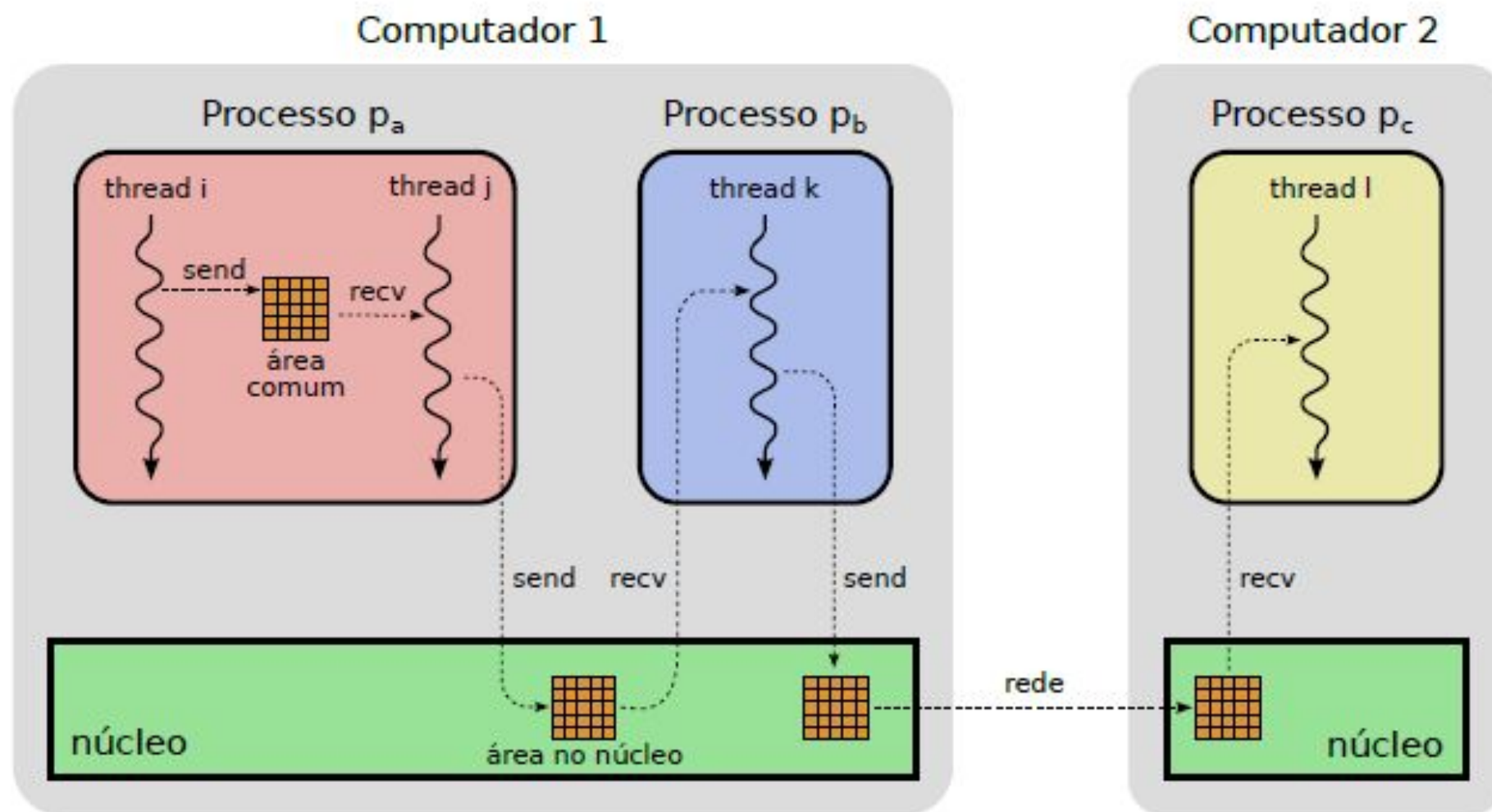
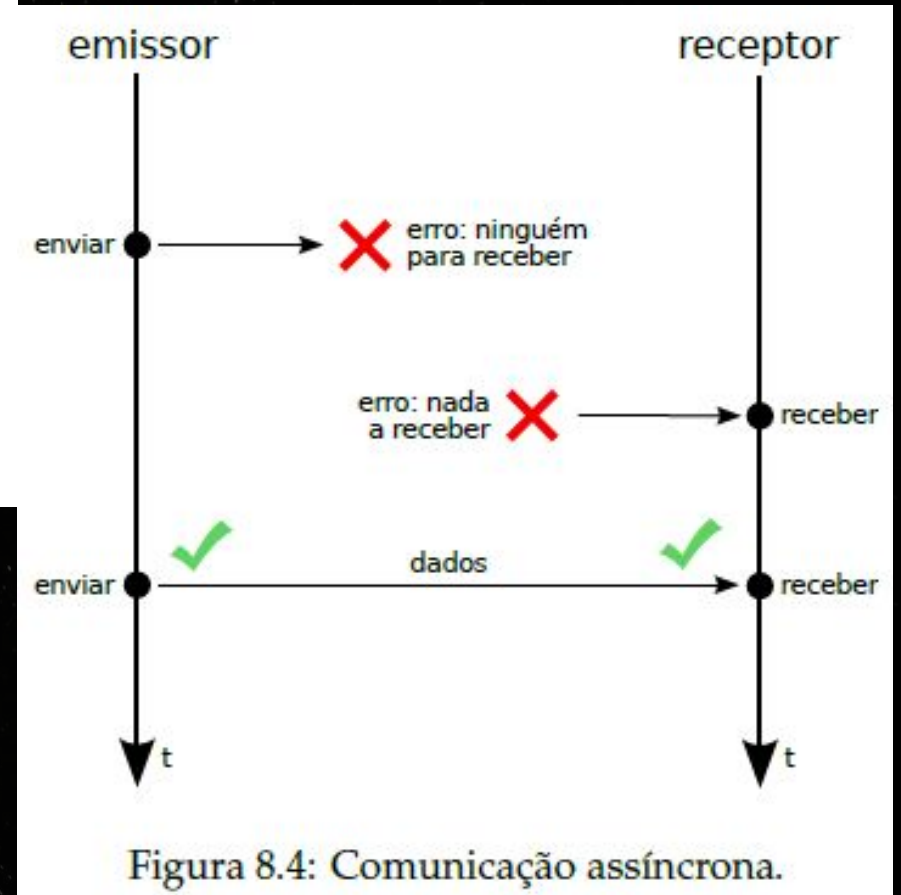
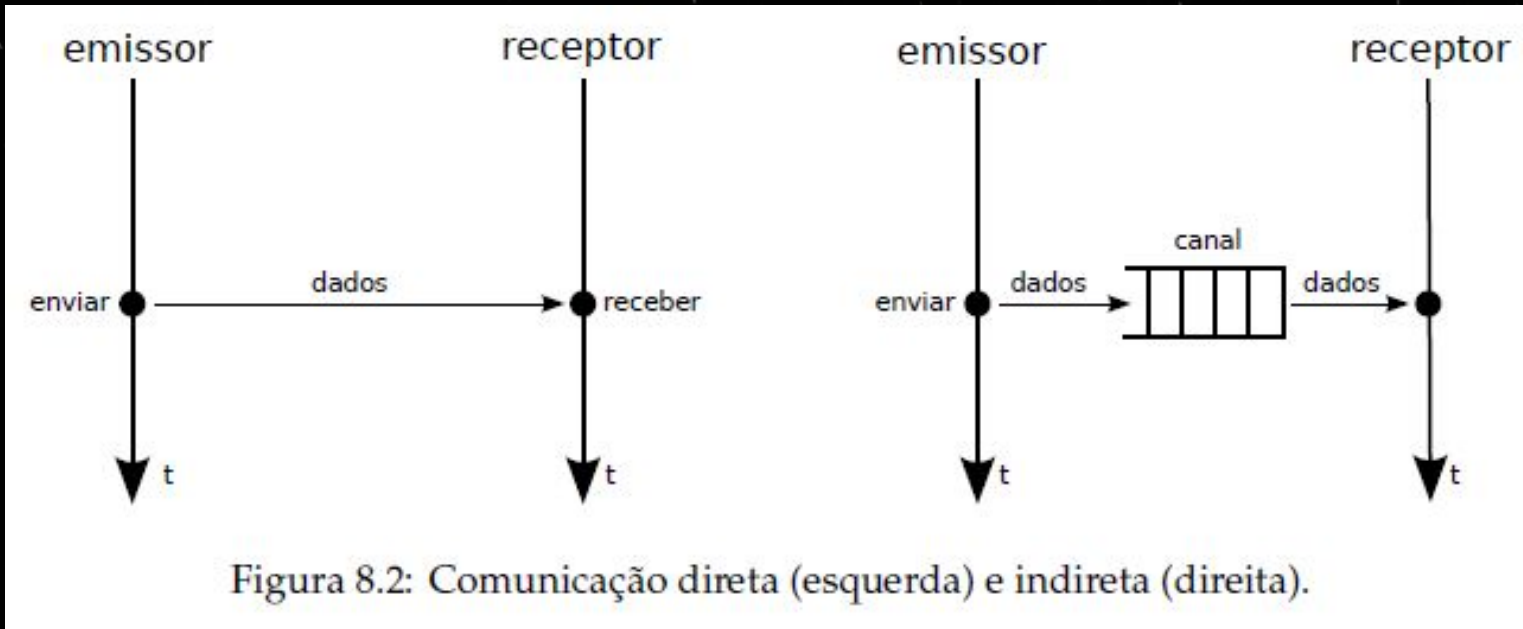


Figura 8.1: Comunicação intraprocesso ($t_i \rightarrow t_j$), interprocessos ($t_j \rightarrow t_k$) e intersistemas ($t_k \rightarrow t_l$).

Formas de Comunicação

Sincronismo



Formato de envio

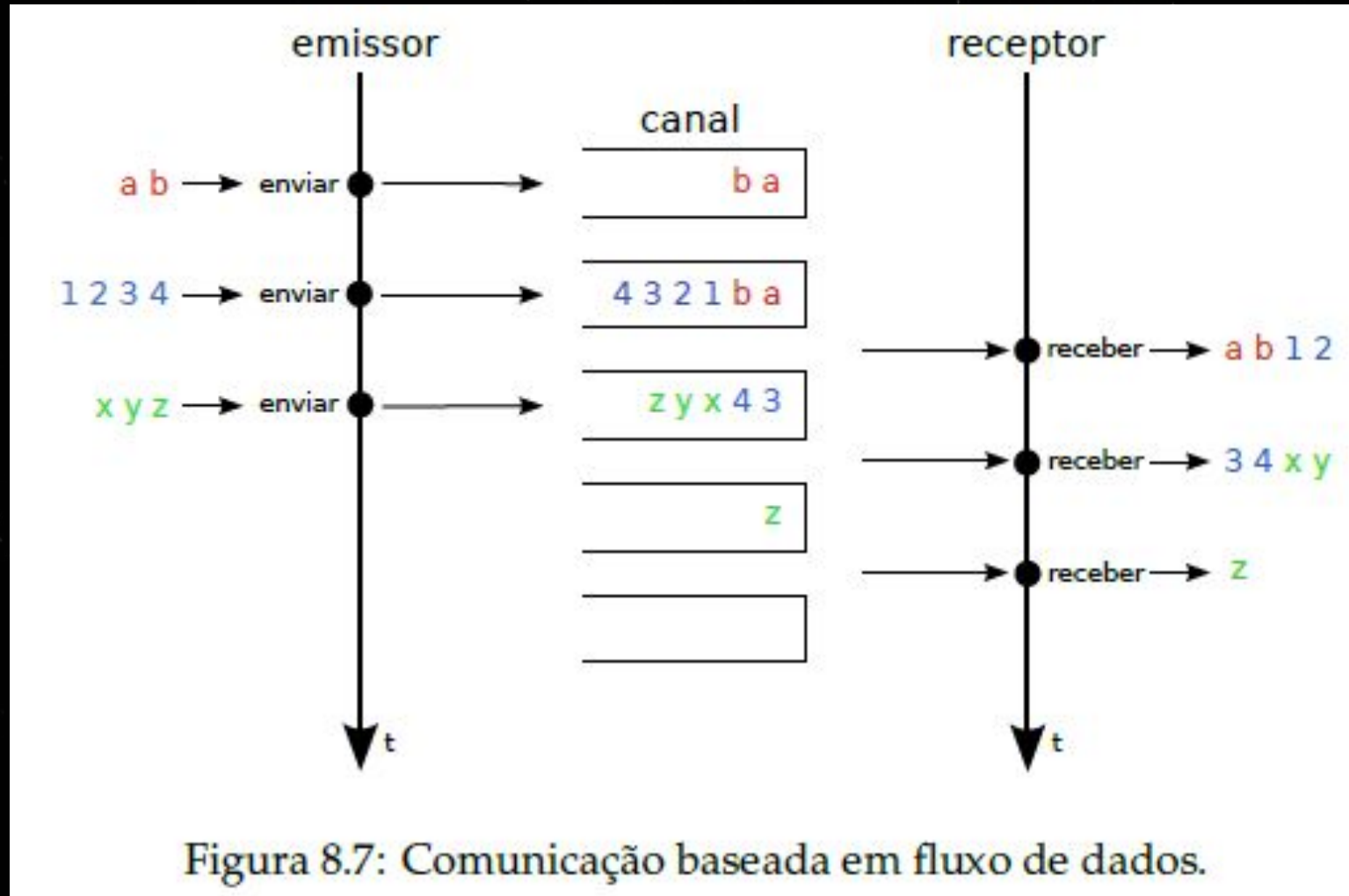
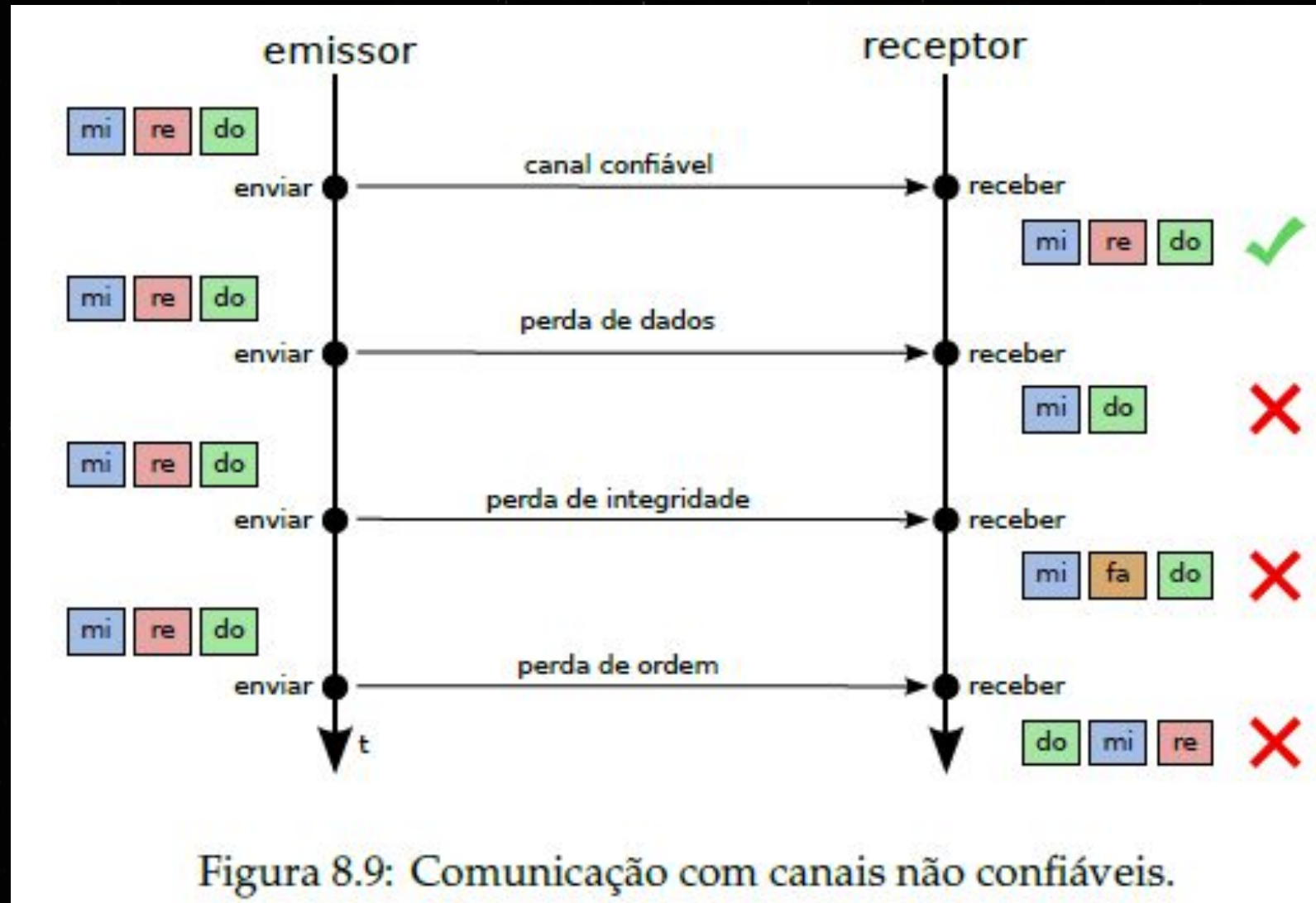
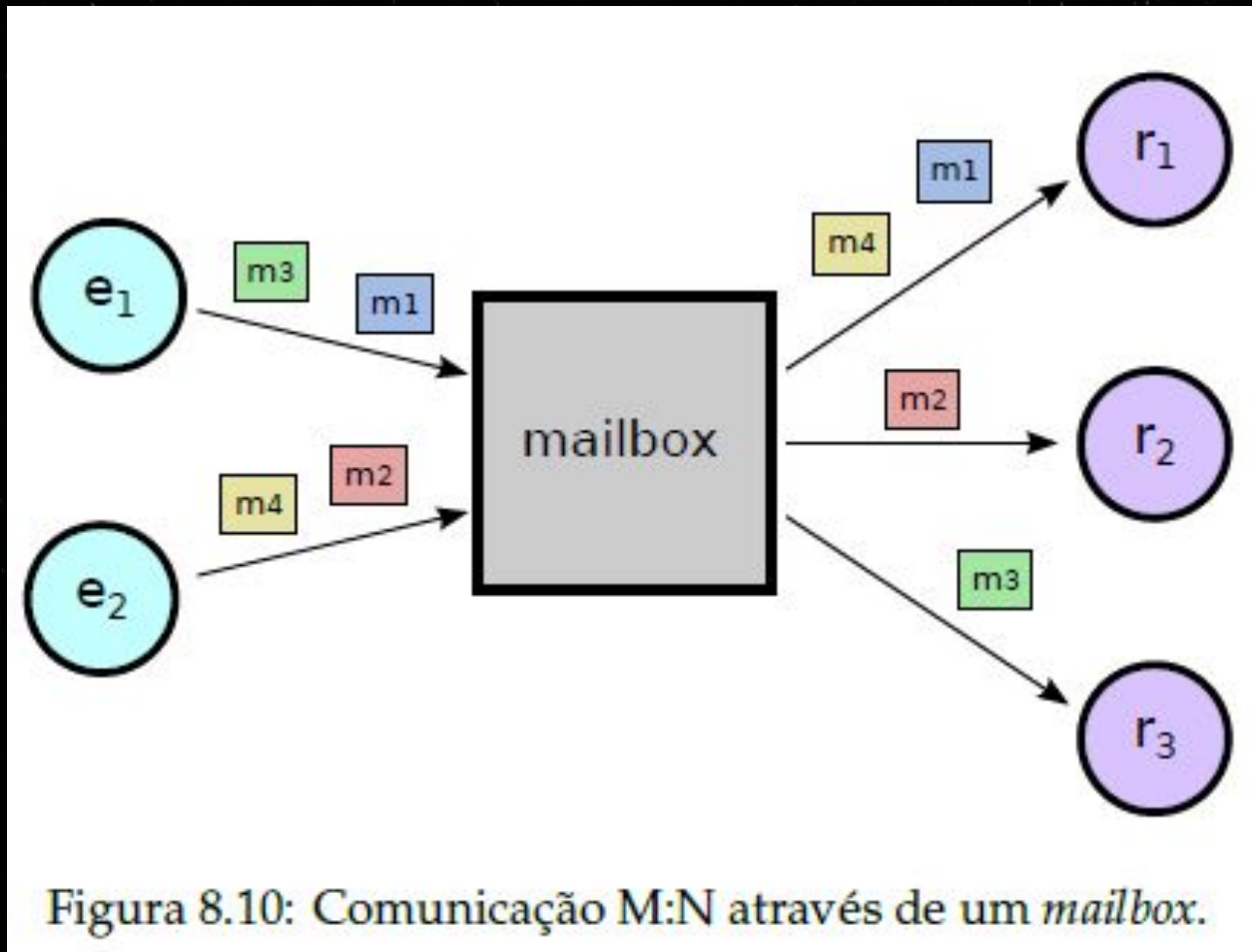


Figura 8.7: Comunicação baseada em fluxo de dados.

Confiabilidade dos canais



Vários participantes da comunicação



Vários participantes da comunicação

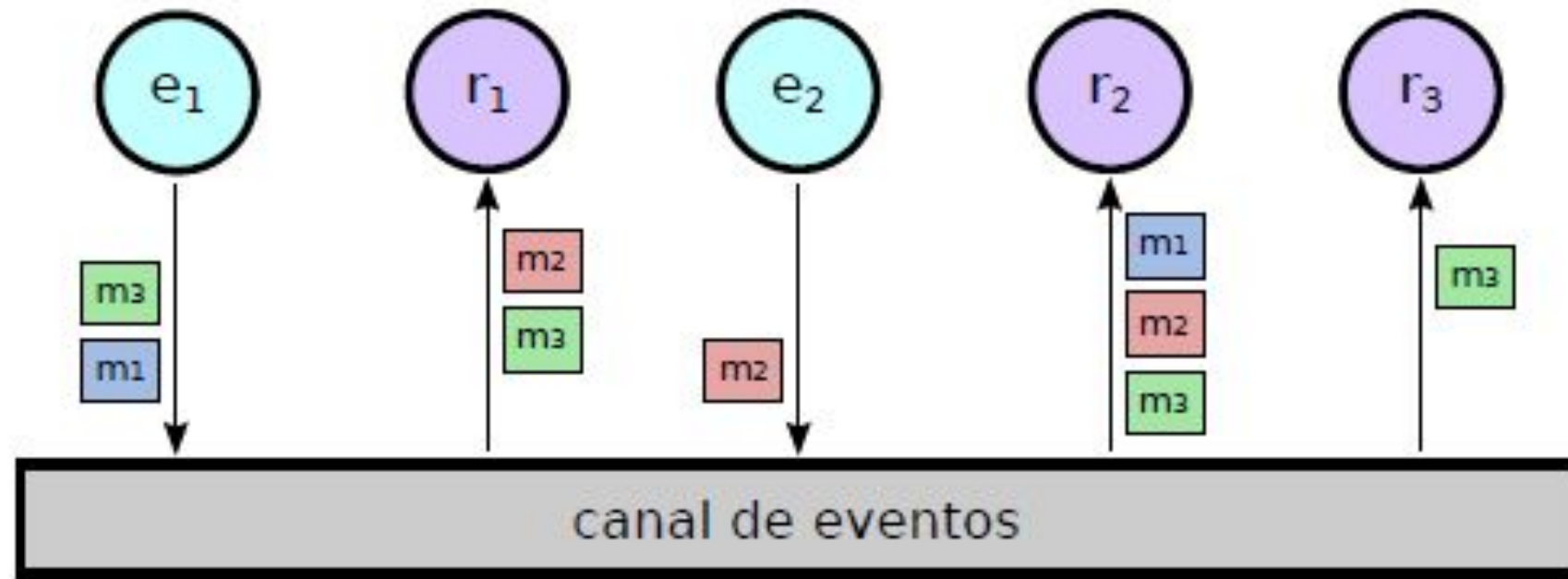


Figura 8.11: Comunicação M:N através de um barramento de mensagens.

Mecanismos de comunicação usados em Sistemas Operacionais

Como é realizado a comunicação?

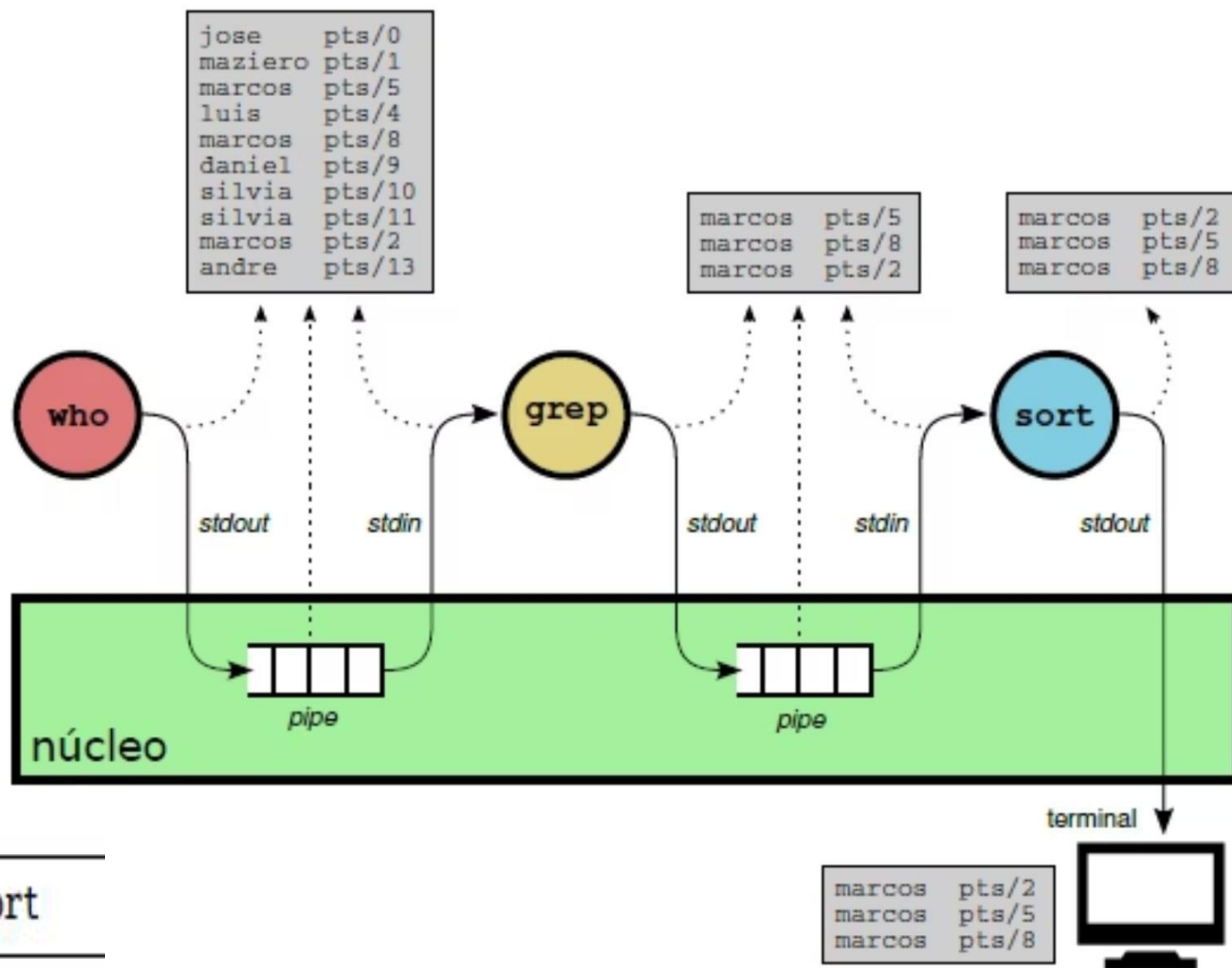
Pipes (em sistemas Unix);

Sockets (Redes);

Memória compartilhada;

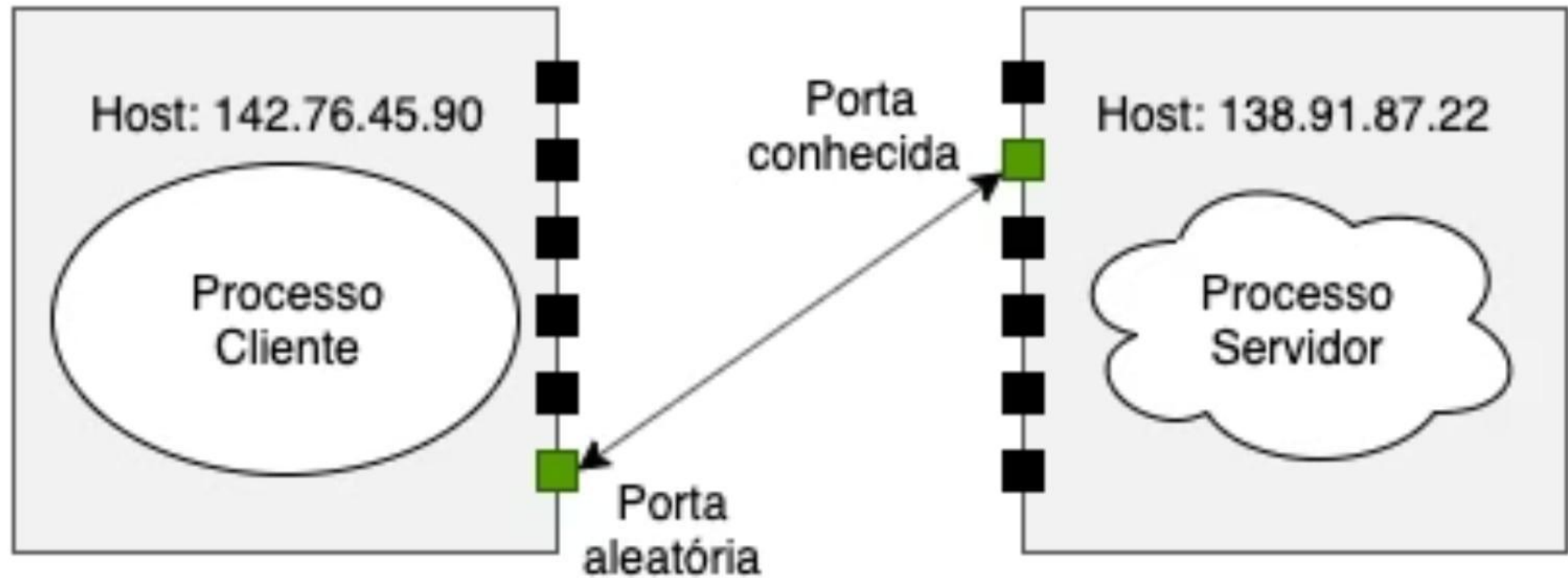
Entre outras formas;

Pipes



```
1 $ who | grep marcos | sort
```

Sockets



Memória Compartilhada

- A comunicação entre tarefas situadas em processos distintos deve ser feita através do núcleo, usando chamadas de sistema;
- Não existe a possibilidade de acesso a variáveis comuns a ambos, pois suas áreas de memória são distintas e isoladas
- Para essas situações, seria conveniente **ter uma área de memória comum que possa ser acessada direta e rapidamente pelos processos interessados**, sem o custo da intermediação do núcleo.

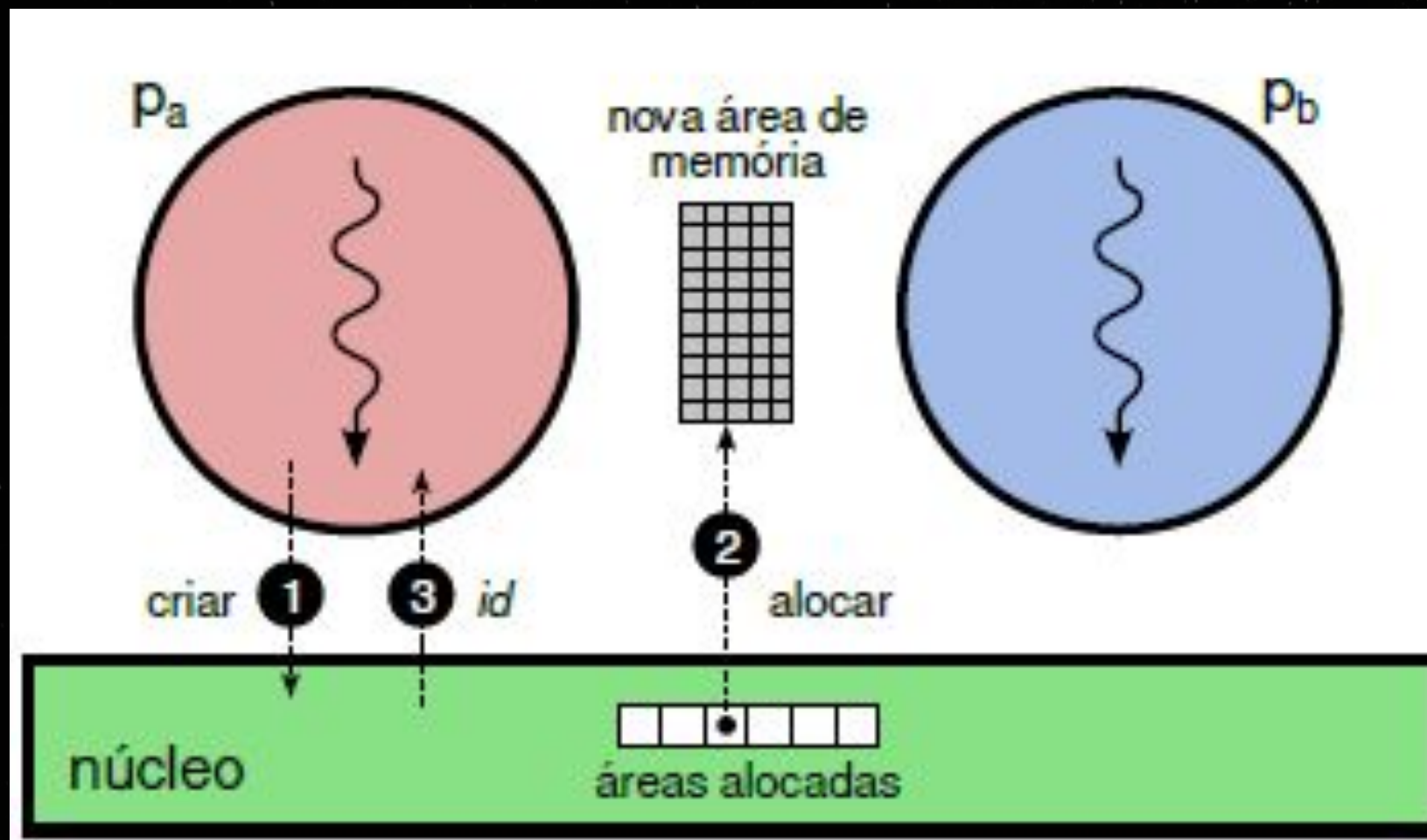
Memória Compartilhada

A maioria dos sistemas operacionais atuais oferece mecanismos para o compartilhamento de áreas de memória entre processos.

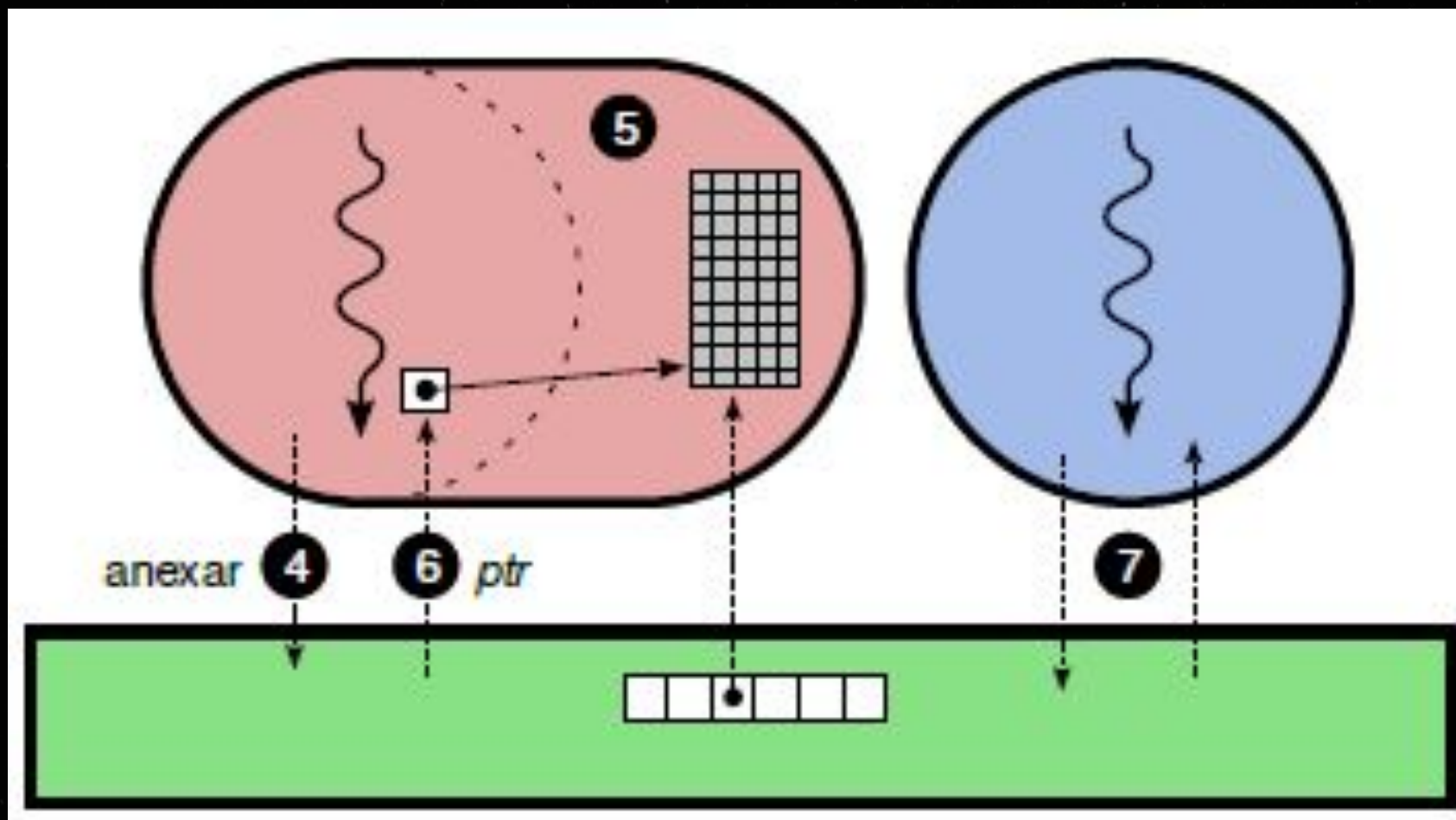
As áreas de memória compartilhadas e os processos que as acessam são gerenciados pelo núcleo, mas o acesso ao conteúdo de cada área é feito diretamente pelos processos, sem intermediação do núcleo.

1. O processo p_a solicita ao núcleo a criação de uma área de memória compartilhada;
2. o núcleo aloca uma nova área de memória e a registra em uma lista de áreas compartilháveis;
3. o núcleo devolve ao processo p_a o identificador (id) da área alocada;
4. o processo p_a solicita ao núcleo que a área identificada por id seja anexada ao seu espaço de endereçamento;
5. o núcleo modifica a configuração de memória do processo p_a para incluir a área indicada por id em seu espaço de endereçamento;
6. o núcleo devolve a p_a um ponteiro para a área alocada;
7. O processo p_b executa os passos 4-6 e também recebe um ponteiro para a área alocada;
8. Os processos p_a e p_b comunicam através de escritas e leituras de valores na área de memória compartilhada.

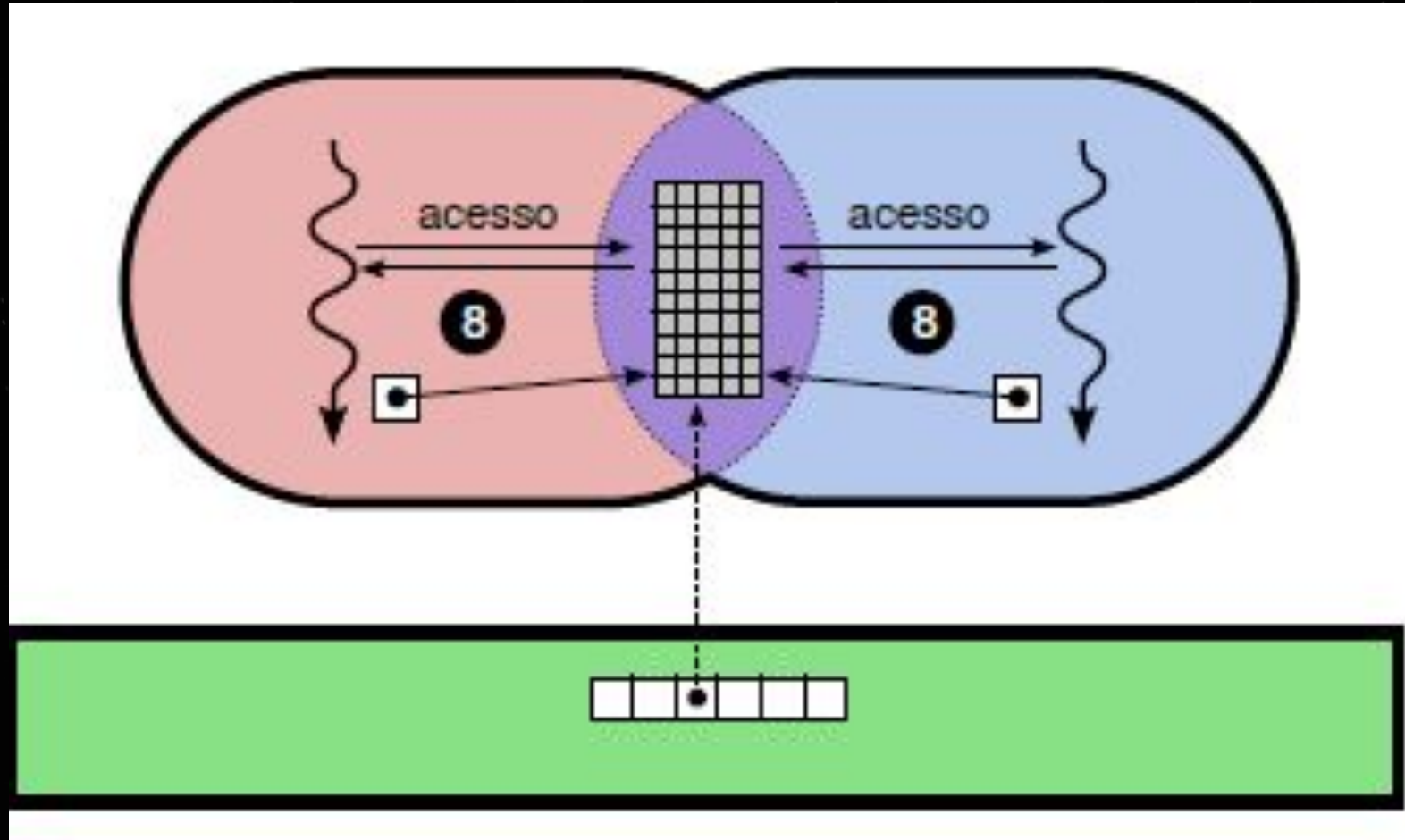
Memória Compartilhada



Memória Compartilhada



Memória Compartilhada



Agora que os dois processos podem acessar a memória ao mesmo tempo, podem-se surgir várias **inconsistências!**

Condições de disputa

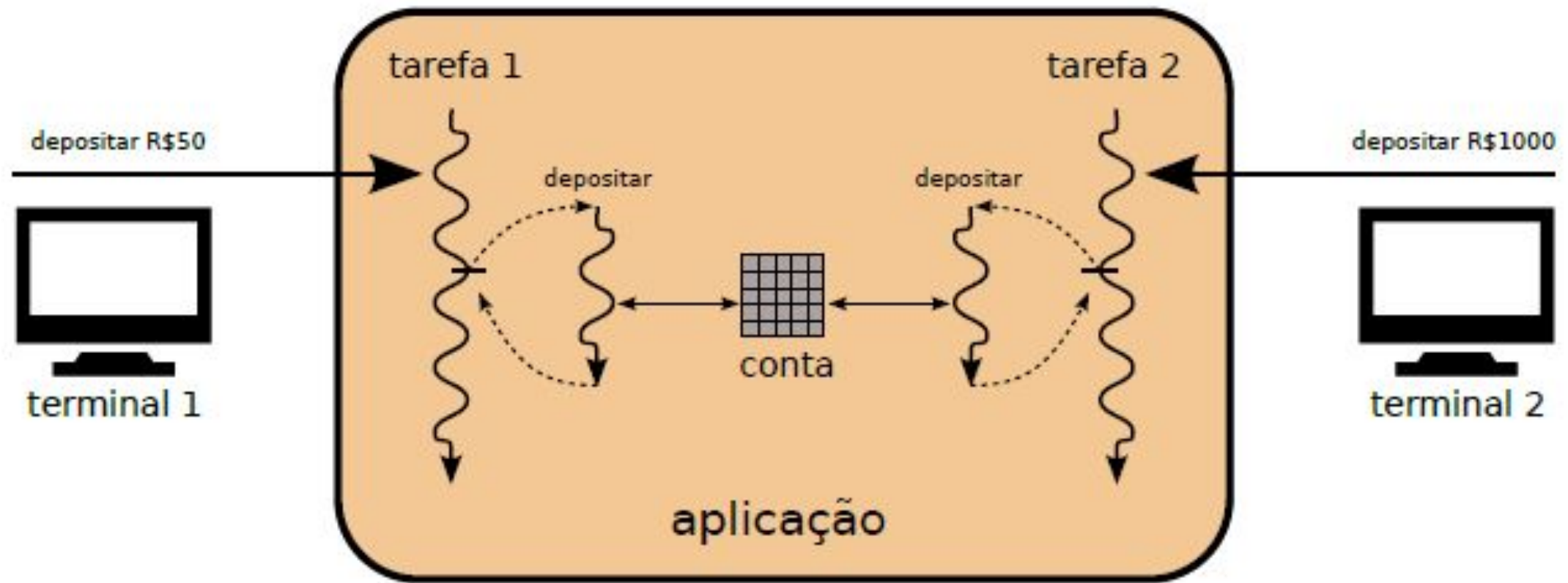


Figura 10.1: Acessos concorrentes a variáveis compartilhadas.

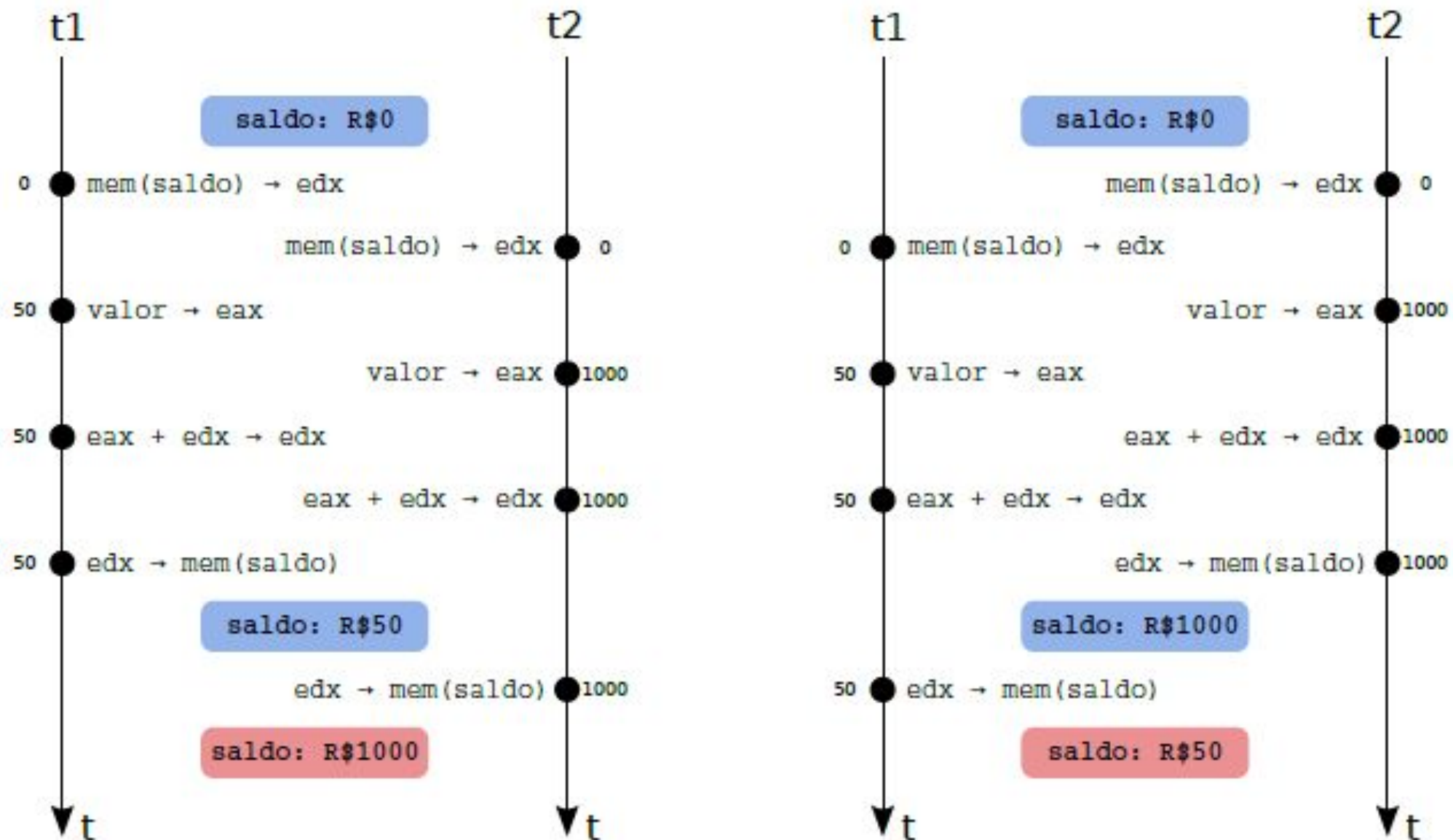


Figura 10.3: Operações de depósito concorrentes.

É importante observar que condições de disputa
são erros dinâmicos, ou seja,
erros que não aparecem no código fonte e que só
se manifestam durante a execução.

contador compartilhado = 1

registro A = 1

registro B = 1

#	Tarefa A	Tarefa B
1	lê contador = 0	—
2	—	lê contador = 0
3	registro A = 1	—
4	—	registro B = 1
5	contador = 1	—
6	—	contador = 1
7	esperado: 2	resultado: 1

Incremento perdido: ocorreram dois incrementos, mas o valor final é 1.

Condição de disputa (ou de corrida)

O que é?

Ocorre quando dois ou mais processos/threads acessam o mesmo dado e pelo menos um deles **modifica** esse dado, e o resultado depende da **ordem de execução**. Podem ocorrer em memória, arquivos, variáveis e etc.



Acesso simultâneo



**Pelo menos um
modifica**



**Resultado depende da
ordem**

Região Crítica

Região crítica

É o trecho do programa que acessa dados ou recursos compartilhados.

No exemplo anterior, a região crítica seria a seguinte:

```
1  (*saldo) += valor ;
```

O que precisa ser protegido?

Não é o programa inteiro

Proteger tudo seria ineficiente e desnecessário.

Apenas o trecho crítico

O trecho que acessa o dado compartilhado.

A operação indivisível

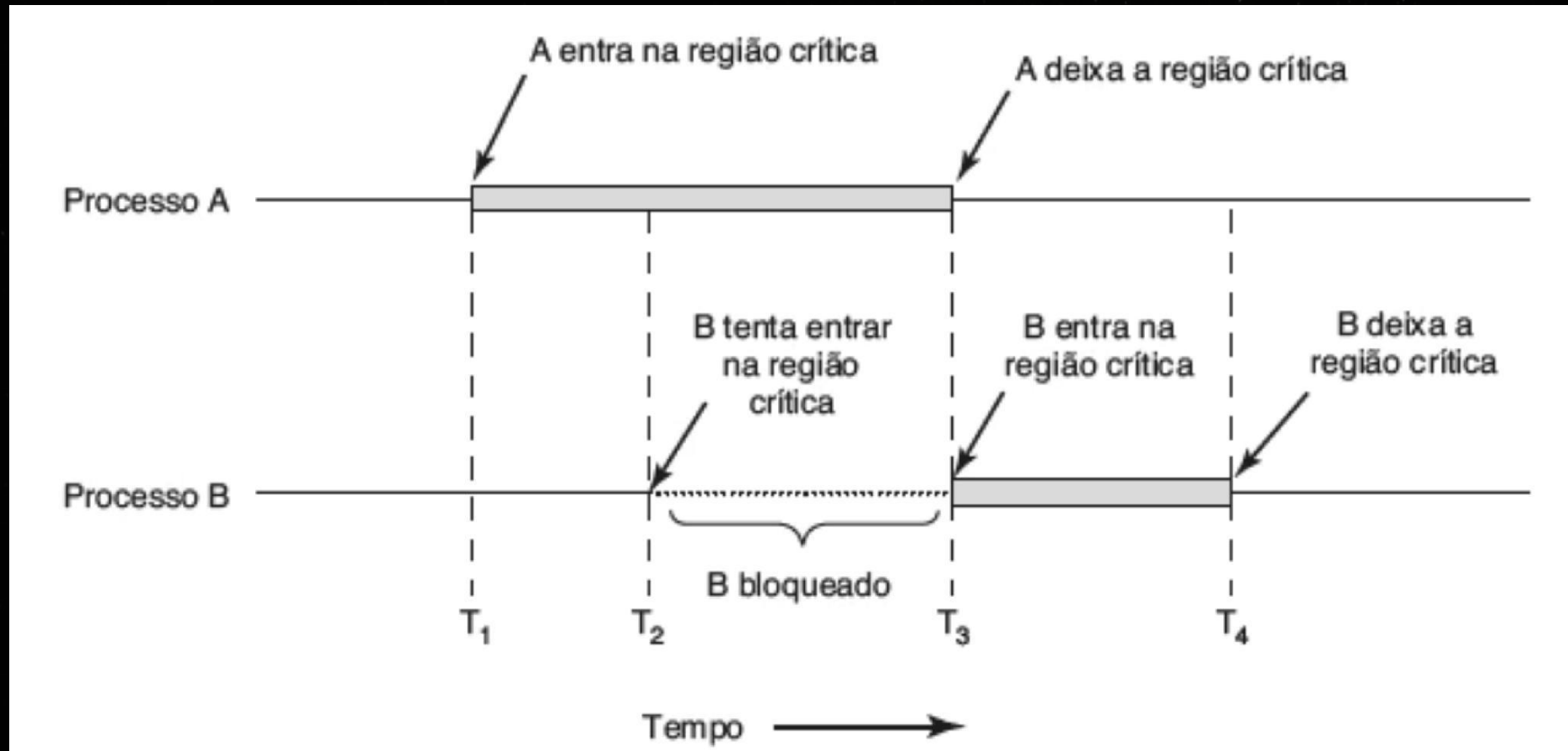
Aquela que não pode ser interrompida no meio.

Onde a interferência causa erro

A parte em que outra thread causaria inconsistência.

A **região crítica** deve ser a menor possível.

Região Crítica



Exclusão mútua

Exclusão mútua

Evitar condições de disputa garantindo acesso exclusivo à região crítica.

MANDAMENTO 1

Nunca dois processos podem estar simultaneamente na região crítica

Somente uma tarefa pode estar dentro da seção crítica em cada instante.

Independência de fatores externos: não se deve assumir nada sobre a execução

A solução deve ser puramente lógica e não depender da velocidade de execução das tarefas, de temporizações, do número de processadores no sistema ou de outros fatores físicos.

Um processo fora da região crítica não deve bloquear outro

A decisão sobre o uso de uma seção crítica deve depender somente das tarefas que estão tentando entrar na mesma.

 **Processo A — fora da região crítica**

A não está usando o recurso compartilhado.

 **Processo B — quer entrar**

B precisa acessar a região crítica agora.

 **Comportamento esperado**

Como A não usa o recurso, A **não deve impedir** que B entre.

Nenhum processo deve esperar eternamente

Uma tarefa que aguarda acesso a uma seção crítica deve ter esse acesso garantido em um tempo finito, ou seja, não pode haver inanição.

Exemplo prático: servidor HTTP

Possíveis regiões críticas em um servidor web:



Cache compartilhada



Contador de acessos



Arquivo de log



Fila de requisições



Pool de conexões

**Soluções para possibilitar o acesso
compartilhado**

Solução trivial com **Espera Ocupada**

```
1 int busy = 0 ;           // a seção está inicialmente livre
2
3 void enter ()
4 {
5     while (busy) {} ;    // espera enquanto a seção estiver ocupada
6     busy = 1 ;           // marca a seção como ocupada
7 }
8
9 void leave ()
10 {
11     busy = 0 ;           // libera a seção (marca como livre)
12 }
```

	Tarefa A	Tarefa B
1	testa busy = 0	—
2	troca de contexto →	—
3	—	testa busy = 0
4	—	busy = 1
5	—	entra na seção crítica
6	← troca de contexto	—
7	busy = 1	—
8	entra na seção crítica	⚠ já está na seção crítica

Não funciona!

Seu grande defeito é que o teste da variável busy (na linha 5) e sua atribuição (na linha 6) são feitos em momentos distintos; caso ocorra uma troca de contexto entre as linhas 5 e 6 do código, poderá ocorrer uma condição de disputa envolvendo a variável busy, que terá como consequência a violação da exclusão mútua: duas ou mais tarefas poderão entrar simultaneamente na seção crítica

Solução com **Alternância de turno**

Outra solução simples para a implementação da exclusão mútua consiste em definir uma variável **turno**, que indica de quem é a vez de entrar na seção crítica. Essa variável deve ser ajustada cada vez que uma tarefa sai da seção crítica, para indicar a próxima tarefa a usá-la.

No exemplo abaixo, suponha que **Processo A** e **Processo B** tenha que acessar a variável compartilhada. Sabemos que **turn** é utilizado para controlar o acesso a região crítica. Suponha também que **turn = 0**...

Variável **Turn** indica de quem é a vez de entrar na região crítica

```
while (TRUE) {  
    while (turn != 0) { // ;  
    critical_region( );  
    turn = 1;  
    noncritical_region( );  
}
```

(a)

Enquanto não é a sua vez, fica verificando de pode entrar

```
while (TRUE) {  
    while (turn != 1) { // ;  
    critical_region( );  
    turn = 0;  
    noncritical_region( );  
}
```

(b)

Se a fica pra sempre aqui, o Processo B nunca consegue acessar a região crítica!

STARVATION!

```
while (TRUE) {  
    while (turn != 0) {}  
    critical_region( );  
    turn = 1;  
    noncritical_region( );  
}
```

(a)

Processo A

```
while (TRUE) {  
    while (turn != 1) { // ;}  
    critical_region( );  
    turn = 0;  
    noncritical_region( );  
}
```

(b)

Processo A

Alternância estrita de turno

A variável `turn` indica quem pode entrar na região crítica.

turn = 0

Estado: **vez de A**

Processo A

```
while (turn != 0) { }
```

```
critical_region();
```

```
turn = 1;
```

```
noncritical_region();
```

Processo B

```
while (turn != 1) { }
```

```
critical_region();
```

```
turn = 0;
```

```
noncritical_region();
```

turn = 0
A pode entrar



A na região crítica



turn = 1
B pode entrar



B na região crítica



turn = 0

B termina e faz `turn = 0`, devolvendo a vez para A.

Ela garante exclusão mútua, mas pode violar a condição de progresso, causando inanição!

- A e B não entram simultaneamente na região crítica.
- Mas existe um problema importante nessa solução: ela obriga os processos a alternarem estritamente.
- Além disso, essa condição também utiliza espera ocupada, consumindo recursos.

Solução de Peterson

Intenção de entrar

Cada processo declara explicitamente que **deseja** entrar na região crítica.

Variável de vez

Uma variável indica de quem é a **vez** de entrar.

Espera condicional

O processo espera apenas se o outro **também quiser entrar** e for a vez do outro.



Processo 0

quer[0] = true
"Quero entrar"

turn = 1
"Dou preferência ao P1"

while (quer[1] && turn == 1)
espera se necessário

 **REGIÃO CRÍTICA**

quer[0] = false
"Terminei"

VARIÁVEIS

quer[0] = **false**
quer[1] = **false**
turn = —

Estado inicial:
ninguém quer
entrar.

Processo 1

quer[1] = true
"Quero entrar"

turn = 0
"Dou preferência ao P0"

while (quer[0] && turn == 0)
espera se necessário

 **REGIÃO CRÍTICA**

quer[1] = false
"Terminei"

Limitação da solução de Peterson

O algoritmo de Peterson foi desenvolvido para garantir a exclusão mútua entre duas tarefas, garantindo também o critério de espera limitada.

Usa espera Ocupada

O processo permanece em laço `while` consumindo CPU.

Desperdício de processamento

CPU ocupada sem realizar trabalho produtivo.

Inversão de prioridade

Um processo de baixa prioridade pode bloquear um de alta prioridade.

Referências bibliográficas

TANENBAUM, Andrew S.; BOS, Herbert.

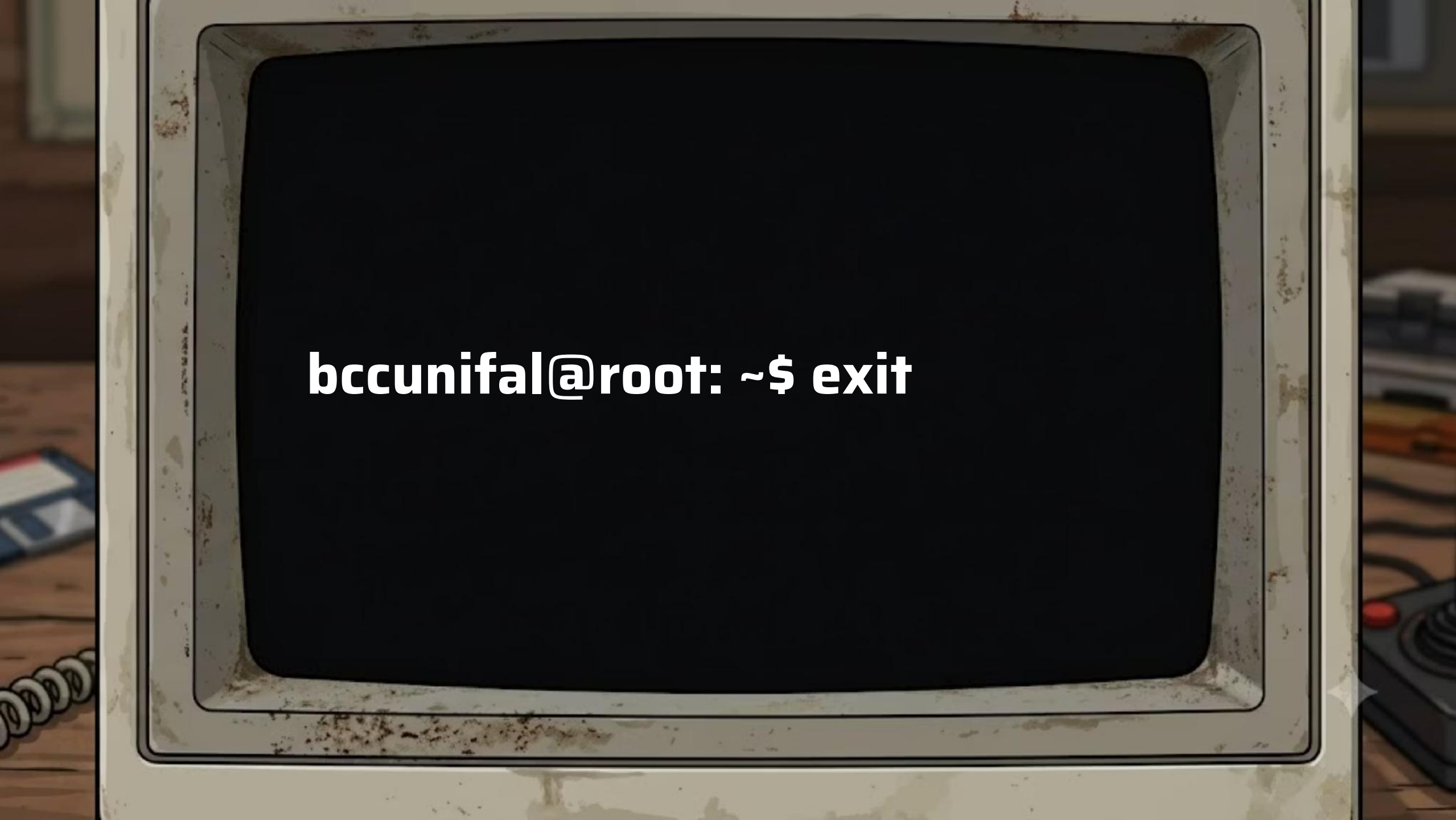
Sistemas Operacionais Modernos. 4. ed. São Paulo: Pearson, 2016.

SILBERSCHATZ, Abraham; GALVIN, Peter B.; GAGNE, Greg.

Fundamentos de Sistemas Operacionais. 9. ed. Rio de Janeiro: LTC, 2015.

MAZIERO, Carlos A.

Sistemas Operacionais: Conceitos e Mecanismos. Curitiba: UFPR, 2019.

A terminal window is displayed on a computer monitor. The monitor has a light-colored, slightly textured bezel. The terminal window itself is a dark rectangle with a thin white border. Inside the terminal, the text 'bccunifal@root: ~\$ exit' is written in a white, monospaced font. The background of the image shows a wooden desk with some blurred objects, including what looks like a keyboard and some papers.

```
bccunifal@root: ~$ exit
```